Domain: Multi-Core / Many-Core Architectures

Distributing Uneven Workloads Across Unequal Multi-Core Machines: A Literature Review

Ahsan Baig — 23L-0515

Sundas Habib — 23L-2580

Department of Computer Science, FAST NUCES, Lahore

Abstract

Modern processors no longer grow faster mainly through higher clock speeds. Instead they add more cores, and many now mix fast and slow cores on the same chip. At the same time, small clusters can be built from everyday laptops, but these machines are rarely equal in power, and their speed can change while they run. This review looks at four recent studies from three related areas: solving games across many computers, sharing work between machines of different speeds, and running parallel programs on chips with mixed core types. We summarise what each study does and finds, then compare them side by side. Reading across them, we identify five research gaps. Most notably, the studies that handle uneven workloads assume equal machines, while the studies that handle unequal machines assume even workloads. The case where both happen at once, on ordinary consumer hardware, has not been examined.

Index Terms— *multi-core processors, heterogeneous computing, load balancing, distributed game solving, hybrid core architecture, work stealing*

I. Introduction

For many years, computers got faster because their processors ran at higher speeds. That approach ran into limits around heat and power, so chip makers turned to adding more cores. Recent chips go further still. Intel's 12th generation processors, for instance, place two kinds of cores on one chip: fast "performance" cores and smaller "efficient" cores that save power.

Clusters have changed in a similar way. Sharing work across computers used to mean a data centre. Now a few ordinary laptops can be linked to work on one problem together. But these laptops are almost never alike. They differ in age, core count, core type and cooling. A laptop may also slow itself down partway through a long job simply because it gets hot.

Some workloads make this harder than others. In game solving, the goal is to work out who wins a game under perfect play. The work here is very uneven: some moves are settled in a second, others take hours, and there is usually no way to tell in advance. Sharing such work fairly is difficult even when machines are identical.

This review looks at four studies published since 2020 that touch on these problems from different sides. Section II explains how they were chosen. Section III reviews each one. Section IV draws out what they have in common, and Section V sets out the research gaps.

II. Review Scope and Method

We searched Google Scholar, arXiv, IEEE Xplore, the ACM Digital Library and SpringerLink using combinations of terms such as *parallel game solving*, *distributed game-tree search*, *heterogeneous load balancing*, *work stealing* and *hybrid core scheduling*. We included studies that were published from 2020 onward and either (a) distributed game-solving work across multiple machines or cores, (b) balanced work across machines of different speeds, or (c) studied parallel scheduling on processors with mixed core types. From these, four studies were selected, two per group member, to cover all three areas.

Table I summarises each study.

TABLE I: Summary of Reviewed Studies

Ref.	Yr.	Method	Workload and setup	Metrics	Performance	Key limitations
[1]	2026	Two-level parallel proof-number search: master hands jobs to workers, each worker uses threads; results kept and shared	Sprouts game; 25 identical server nodes, 10 Gbit/s Ethernet	Speedup, repeated work, worker use	208.6× speedup on 1024 cores (332.9× with a simple heuristic); 42 new positions solved	Equal machines only; settings tuned for one cluster
[2]	2023	Manager–worker game solver with a neural network guide that is retrained during the solve	16 openings of 7x7 Killall-Go; 3 identical GPU servers, 384 workers	Positions checked, time taken	21.69% of positions and 23.54% of time vs. baseline	Equal high-end servers; aims to cut search, not split it fairly
[3]	2024	Adaptive work stealing (A2WS): nodes share speed information and take the right amount of work	Seismic simulation; identical supercomputer nodes with some cores switched off	Runtime gain	About 10% better than two other methods at 128 nodes	Speed differences faked and fixed; even task sizes
[4]	2026	Three OpenMP versions: equal split, speed-measured split, speed-measured plus adaptive split	Monte Carlo π estimation; one Intel i5-1235U laptop	Runtime, scalability	Speed-measured versions clearly beat the equal split	One laptop; very even workload

III. Review of Selected Studies

A. Solving Games Across Many Machines

Čížek, Balko and Schmid [1] take on a long-standing problem with proof-number search, one of the most common methods for solving games: older parallel versions stopped speeding up after a few dozen cores. Their answer splits the work at two levels. A master program keeps the overall search and hands jobs to worker programs on other machines, and each worker shares its job among several threads on its own machine. Workers keep what they learn between jobs, and useful results are passed to the others.

On the game Sprouts, which has very lopsided game trees, the method reached a 208.6× speedup on 1024 cores without any game-specific tricks. The best earlier method managed 21.3× in the same setting. Adding a simple rule for choosing which moves to check first pushed the speedup to 332.9×, and the solver settled 42 Sprouts positions nobody had solved before. One result stands out: when workers threw away their results after each job, they

ended up solving the same positions again and again, which slowed everything down badly. Keeping and sharing results removed much of that waste.

Wu et al. [2] built a game solver around a manager and many workers. The manager holds the search tree and uses a trained neural network to decide which positions to send out, and workers solve what they receive and report back. The new part is a trainer that runs alongside the solve and keeps improving the network, using positions that have just been solved or look important. This matters because a network trained on good play makes poor guesses on the odd positions a complete solver must check.

Tested on 16 hard openings of 7x7 Killall-Go, a smaller version of Go, the system checked only 21.69% of the positions and used 23.54% of the time compared with the same solver without the trainer. The savings grew as the problems got harder.

B. Sharing Work Between Machines of Different Speeds

Fernandes et al. [3] study a familiar problem: when machines of different speeds share work, the fast ones finish early and wait around for the slow ones. Their method, A2WS, has each machine track how long its tasks take on average and how many it has left, and pass this to its neighbours. From that, a machine can work out how many tasks it ought to have for its speed and "steal" the difference from a neighbour holding too many. Stealing begins as soon as a machine finishes its first task, instead of waiting until it runs dry.

On a seismic simulation, A2WS did about 10% better than two other methods at 128 machines. It struggled, though, when each machine had only a few tasks, because there was not enough time for balancing to take effect.

C. Scheduling on Chips with Fast and Slow Cores

Beserra, Moreno and Araujo [4] ask whether standard parallel programming tools still work well on chips that mix fast and slow cores, given that most of these tools quietly assume every core runs at the same speed. They wrote three versions of a program that estimates π using random numbers. One splits the work equally. One measures each core's speed first and splits the work to match. The third measures speed and also adjusts how work is handed out while the program runs.

On an Intel Core i5-1235U laptop chip, the equal split left some cores waiting on others. Both speed-measuring versions ran clearly faster and scaled better.

IV. Synthesis

Reading the four studies together, a pattern appears. They split neatly into two camps. The game-solving studies [1], [2] deal with very uneven work but run on identical, powerful machines. The other two [3], [4] deal with machines or cores of different speeds but only with even, predictable work. Each camp solves half of a harder problem.

These halves also interact. When task sizes are unpredictable, a slow machine can end up holding one very large task near the end, leaving faster machines idle. When machine speed drifts during a run, any speed measured at the start slowly becomes wrong. And the more

finely work is split to balance it, the more likely machines are to repeat each other's work. None of the reviewed studies faces all of these pressures at once.

V. Research Gaps

Gap 1: Real differences between machines are avoided or faked. Both game-solving studies use identical hardware: 25 identical server nodes in [1] and three identical GPU servers in [2]. Study [3] does look at machines of different speeds, but only by switching off cores on otherwise identical supercomputer nodes. Study [4] covers fast and slow cores, but only inside a single laptop. No study tests a truly mixed group of machines, differing in processor generation, core design, memory and cooling, and none looks at differences both between machines and inside each chip at the same time.

Gap 2: Machine speed is treated as fixed. In [3], each machine's speed stays exactly the same for the whole run, because it comes from a fixed number of cores. In [4], core speeds are measured before the work begins, and that measurement can go out of date if the chip slows down as it heats up. Study [1] ran long jobs but did not look at speed changing over time. Laptops boost at the start, slow down when hot, and move work between fast and slow cores, and how this drift affects work sharing over a long run has not been studied.

Gap 3: Uneven work and unequal machines have not been studied together. The game-solving studies [1], [2] handle very uneven task sizes, but only on equal machines. Study [3] handles unequal machines but uses tasks of similar size, and its authors leave uneven task sizes to future work. Study [4] uses a workload where every piece costs about the same. Study [3] also performed poorly when each machine had only a few tasks, which is exactly the situation in a small cluster. The combination of unpredictable task sizes and unequal machines remains open.

Gap 4: Everyday hardware and networks are left out. The reviewed systems run on data-centre servers with a 10 Gbit/s network [1], GPU servers [2], and a supercomputer [3]. The method in [2] also relies heavily on graphics cards, and the settings in [1] were tuned by testing every option on one particular cluster, so it is unclear how either would carry over to ordinary machines. Whether uneven work can be shared well across a few consumer laptops, over home Wi-Fi or a plain cable, has not been tested.

Gap 5: Repeated work in small, unequal setups is not measured. Study [1] shows that solving the same positions twice seriously hurts performance, and that keeping and sharing results helps. But this was measured on a large cluster of equal machines, and [2] focuses on checking fewer positions overall rather than on how the work is divided. How much work gets repeated when a problem is split between only a couple of unequal machines, and how the size of each split affects this, is still unknown.

Taken together, these gaps point toward a common open question: how to share unpredictable, uneven workloads efficiently across a small group of unequal consumer machines, where speed differs both between machines and within each chip and changes while the work runs. Our next phase will narrow this into a specific problem statement and approach.

References

[1] T. Čížek, M. Balko, and M. Schmid, "Massively parallel proof-number search for impartial games and beyond," in *Proc. AAAI Conf. Artificial Intelligence (AAAI)*, 2026. [Online]. Available: https://arxiv.org/abs/2511.10339

[2] T.-R. Wu, H. Guei, T.-H. Wei, C.-C. Shih, J.-T. Chin, and I-C. Wu, "Game solving with online fine-tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2311.07178

[3] J. B. Fernandes, Í. A. S. de Assis, I. M. S. Martins, T. Barros, and S. Xavier-de-Souza, "Adaptive asynchronous work-stealing for distributed load-balancing in heterogeneous systems," *arXiv preprint* arXiv:2401.04494, 2024. [Online]. Available: https://arxiv.org/abs/2401.04494

[4] D. Beserra, E. D. Moreno, and J. Araujo, "Evaluating OpenMP scheduling and calibration strategies on asymmetric multicore processors: A Monte Carlo case study," in *Advanced Information Networking and Applications (AINA 2026)*, Lecture Notes on Data Engineering and Communications Technologies, vol. 294. Cham, Switzerland: Springer, 2026, pp. 357–369, doi: 10.1007/978-3-032-23260-1_30.